Assignment 6 Report: Medians, Order Statistics, and Elementary Data Structures

Anushka Nanaware

STUDENT ID: 005042356

Department of Computer Science

University of the Cumberlands

MSCS532 - M80 Algorithms and Data Structures

Dr. Micahel Solomon

March 13, 2026

GITHUB LINK: https://github.com/ananaware/-Medians-Order-Statistics-and-Data-Structures-MSCS532-ADS-Assignment6

Algorithms for finding medians and order statistics are important concepts in computer science. An order statistic refers to the element that would appear at a specific position if the data were sorted. For example, the smallest element is the first order statistic, while the median represents the middle value of the dataset. Efficient selection algorithms are useful because they allow us to find these values without fully sorting the entire dataset.

According to Cormen et al. (2022), selection algorithms can determine the *kthk^{th}*kth smallest element of a list in linear time. Two common approaches are randomized selection algorithms and deterministic selection algorithms. Randomized algorithms are usually faster in practice, while deterministic algorithms guarantee linear time even in the worst case.

The implementation of Randomized **Quickselect** and **Deterministic Selection was done with the use of the Median of Medians method** along with the implementation of various fundamental data structures was done that too form scratch which included linked lists, stacks and queues. The performance of the algorithms was also tested experimentally to observe their behavior on different input sizes.

Part 1: Implementation of Selection Algorithms

Randomized Quickselect Algorithm

The Randomized Quickselect algorithm is a variation of the QuickSort algorithm. Instead of sorting the entire array, the algorithm partitions the array around a randomly chosen pivot element and then recursively searches only the relevant portion of the array and these are the steps:

- The selection of a pivot is randomly done away from the array.
- Partition the array into three groups:
    - elements that are smaller, equal and greater than the pivot
- The order statistic that is desired should be decided which partition has it.
- Recursively apply the algorithm only to that partition.

This method avoids sorting unnecessary elements, which significantly improves performance compared to full sorting. According to Cormen et al. (2022), the expected running time of Randomized Quickselect is **O(n)** because random pivots typically divide the array into reasonably balanced partitions.

Deterministic Selection (Median of Medians)

This algorithm makes sure that even in the worst possible case the running times stay stable and linear and selects a good pivot using the **median of medians strategy**. The process works as follows:

- The array is divided into groups which consist of at least 5 elements.
- Then the median of each group is calculated.
- Ensure that the median of these medians is getting computed recursively.
- Then the value that we found in the previous steps is sued as the pivot.
- Partition the array around the pivot and continue searching in the relevant partition.

Cormen et al. (2022) show that this method guarantees a balanced partitioning of elements. As a result, the algorithm always runs in **O(n)** time, even in the worst case.

However, the algorithm has additional overhead because it performs more calculations to determine a good pivot. This often makes it slower than randomized algorithms in practice.

Program Execution

The implemented algorithms were tested using Python to verify their correctness. The program allows the user to input numbers and select a value for *kk*k. The program then calculates the *kthk^{th}*kth smallest element using both algorithms.

```
C:\Users\anush\Documents\MSCS532_Assignment6>python selection_algorithms.py
Array: [12, 3, 5, 7, 4, 19, 26, 8, 15]
k = 4
Randomized Quickselect result: 8
Deterministic Select result: 8

C:\Users\anush\Documents\MSCS532_Assignment6>
```

This screenshot shows the execution of the program with a sample input array. Both algorithms return the same result, confirming that the implementations are correct.

Custom Input Test

The program was also tested with user-provided input values to ensure that the algorithm works correctly with different datasets.

```
C:\Users\anush\Documents\MSCS532_Assignment6>notepad selection_algorithms.py


C:\Users\anush\Documents\MSCS532_Assignment6>python selection_algorithms.py
Enter numbers separated by space: 10 4 7 2 15 9 1
Enter k (0-based index): 3
Array: [10, 4, 7, 2, 15, 9, 1]
k = 3
Randomized Quickselect result: 7
Deterministic Select result: 7

C:\Users\anush\Documents\MSCS532_Assignment6>
```

This screenshot demonstrates that the program correctly calculates the selected order statistic based on user input.

Time Complexity Analysis

The time complexity of both algorithms can be analyzed using algorithmic theory.

Randomized Quickselect

The expected running time is *O(n)O(n)*O(n), because each partition typically divides the array into smaller subproblems. However, in rare cases, the pivot may be poorly chosen, which could lead to a worst-case time complexity of: *O(n2)O(n^2)*O(n2)

Deterministic Selection

The deterministic selection algorithm guarantees that a significant portion of elements are removed in each step. Because of this guarantee, the recurrence relation leads to a worst-case time complexity of: *O(n)O(n)*O(n)

Space Complexity

Both algorithms use recursion and additional lists for partitioning. Therefore, the space complexity is approximately: *O(n)O(n)*O(n) due to temporary storage and recursive calls.

Empirical Performance Analysis

To observe the practical performance of both algorithms, experiments were conducted using randomly generated arrays of different sizes. The input sizes were tested are 100, 1000, 5000, and 10000. Execution time was measured for both algorithms.

```
C:\Users\anush\Documents\MSCS532_Assignment6>notepad experiment.py

C:\Users\anush\Documents\MSCS532_Assignment6>python experiment.py
Size | Randomized Quickselect | Deterministic Select
-----------------------------------------------------
100 | 0.000055 seconds | 0.000054 seconds
1000 | 0.000216 seconds | 0.000468 seconds
5000 | 0.001187 seconds | 0.001628 seconds
10000 | 0.002611 seconds | 0.004637 seconds

C:\Users\anush\Documents\MSCS532_Assignment6>
```

Observations

The experimental results show that Randomized Quickselect generally runs faster than the deterministic algorithm. This happens because the deterministic method performs additional steps to calculate the median of medians. Although the deterministic algorithm guarantees worst-case linear time, the randomized algorithm is usually more efficient in real-world scenarios.

Part 2: Elementary Data Structures

In addition to selection algorithms, several basic data structures were implemented. These data structures are fundamental building blocks used in many software systems.

Stack

The **Last-In-First-Out (LIFO)** principle is followed by a stack which means that the first element which is removed is the last element that is put in the stack and the following operations were implemented:

- push
- pop
- peek
- is_empty

Time Complexity

Operation	Complexity
Push	O(1)
Pop	O(1)
Peek	O(1)

Stacks are commonly used for:

- function call management

- expression evaluation
- undo operations in software applications

Queue

A queue follows the **First-In-First-Out (FIFO)** principle, which means that the first element which is removed is the first element that is added. The operations implemented include:

- enqueue
- dequeue

Time Complexity

Operation	Complexity
Enqueue	O(1)
Dequeue	O(n) using array implementation

Queues are used in many systems such as:

- task scheduling
- message queues
- network packet processing.

Singly Linked List

This is a data structure which is very much dynamic and has various nodes and each of these nodes consist of data and a pointer that points to the next node and for this the following operations were implemented:

- insertion
- deletion
- Traversal

Operation	Complexity
Insertion	O(n)
Deletion	O(n)
Traversal	O(n)

Time Complexity

Linked lists allow efficient insertion and deletion compared to arrays because elements do not need to be shifted in memory.

Data Structure Program Output

The data structure implementations were tested to verify their correctness. This below output confirms that stack, queue, and linked list operations work correctly.

```
C:\Users\anush\Documents\MSCS532_Assignment6>notepad data_structures.py

C:\Users\anush\Documents\MSCS532_Assignment6>python data_structures.py
----- STACK TEST -----
Stack: [10, 20, 30]
Popped: 30
Stack: [10, 20]

----- QUEUE TEST -----
Queue: [5, 15, 25]
Dequeued: 5
Queue: [15, 25]

----- LINKED LIST TEST -----
Linked List: [1, 2, 3]
Linked List: [1, 3]

C:\Users\anush\Documents\MSCS532_Assignment6>
```

This assignment explored important algorithmic concepts related to selection algorithms and basic data structures. The theoretical analysis showed that both algorithms can achieve linear time complexity, but the deterministic algorithm guarantees this performance even in the worst case. The experimental results confirmed that Randomized Quickselect typically performs faster in practice due to lower overhead.

In addition to algorithm implementation, several fundamental data structures were developed from scratch, including stacks, queues, and linked lists. Overall, this assignment provided both theoretical understanding and practical experience with key algorithmic techniques that are widely used in computer science and software engineering.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.